



St. Xavier's College (Autonomous), Ahmedabad

Department of Computer Sciences

Academic Year : 2026-27

Practical Assignment

Course : Bachelor in Computer Applications
Year : Second Year
Semester – Division : 3 – A
Subject : Data structure with c++ (Practical)
Code : BCAMC333L

Submitted by:

Student Name :
DHRUVI S. SAGAR
Roll No. : 25-BCA-060

Submitted to :

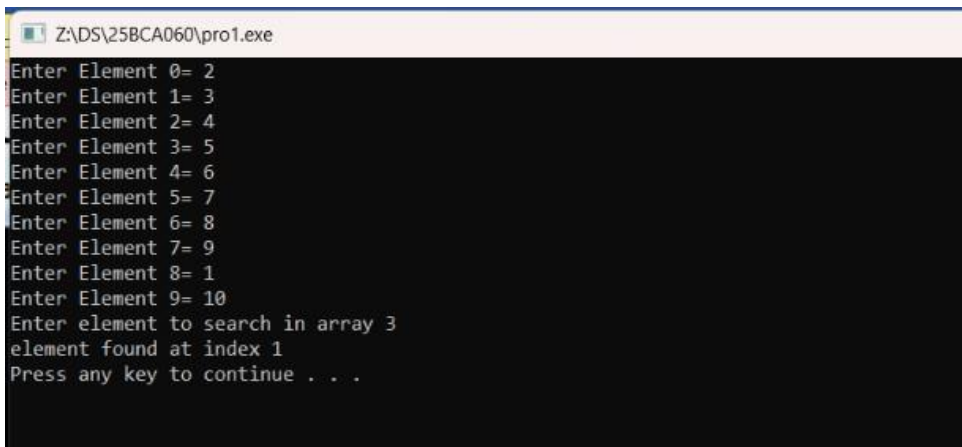
Signature

Date of submission

Student Signature

Program-1.// Write a program in C++ for Linear Search / Sequential Search.

```
#include<iostream>
using namespace std;
int main() {
    int a[10],i,f=0,element,index;
    for(i=0;i<10;i++) {
        cout << "Enter Element " << i << " = ";
        cin >> a[i];
    }
    cout << "Enter element to search in array ";
    cin >> element;
    for(i=0;i<10;i++) {
        if(a[i]==element) {
            f=1;
            index=i;
        }
    }
    if(f==1)
        cout << "element found at index " << index << endl;
    else
        cout << "element is not found" << endl;
    system("pause");
    return 0;
}
```

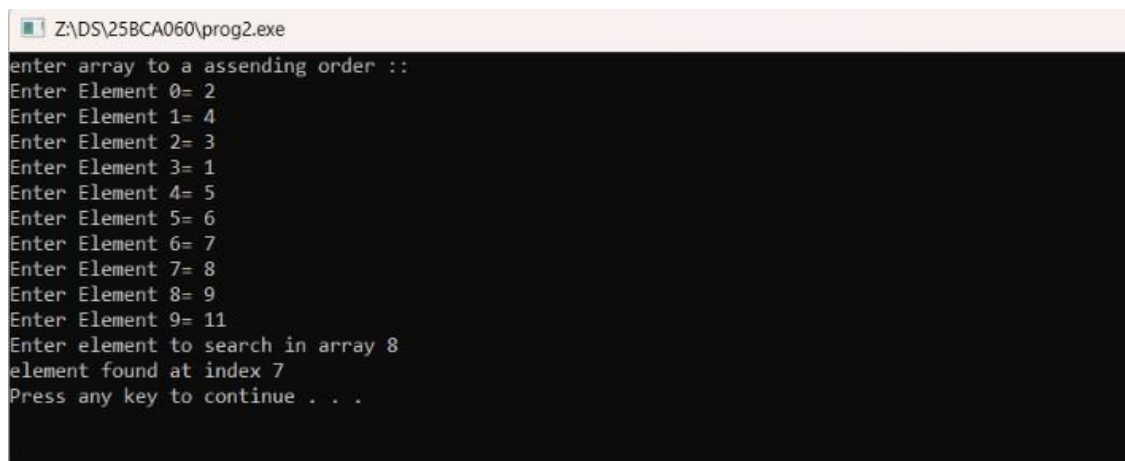
OUTPUT:

```
Z:\DS\25BCA060\pro1.exe
Enter Element 0= 2
Enter Element 1= 3
Enter Element 2= 4
Enter Element 3= 5
Enter Element 4= 6
Enter Element 5= 7
Enter Element 6= 8
Enter Element 7= 9
Enter Element 8= 1
Enter Element 9= 10
Enter element to search in array 3
element found at index 1
Press any key to continue . . .
```

Program-2.// Write a program in C++ for Binary Search .

```
#include<iostream>
using namespace std;
int main() {
    int a[10],i,f=0,element,index,beg,end,mid;
    cout << "enter array to a assending order :: "<< endl;
    for(i=0;i<10;i++) {
        cout << "Enter Element " << i << " = ";
        cin >> a[i];
    }
    cout << "Enter element to search in array ";
    cin >> element;
    beg=0;
    end=9;
```

```
while(beg <= end) {
    mid = (beg+end)/2;
    if(a[mid] == element) {
        f=1;
        index = mid;
    }
    if(a[mid] > element)
        end = mid-1;
    else
        beg = mid+1;
}
if(f==1)
    cout << "element found at index " << index << endl;
else
    cout << "element is not found" << endl;
system("pause");
return 0;
}
```

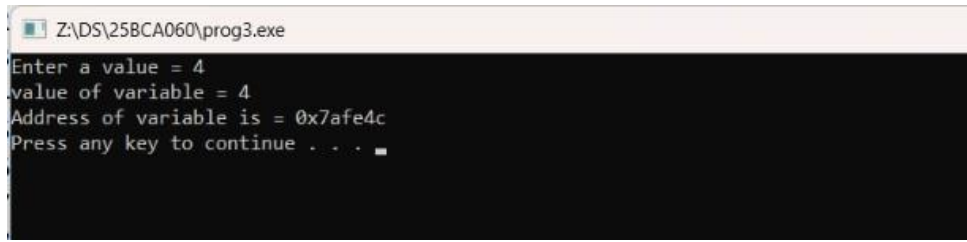
OUTPUT:A screenshot of a Windows command prompt window titled "Z:\DS\25BCA060\prog2.exe". The program prompts the user to "enter array to a assending order ::". The user enters 10 elements: 2, 4, 3, 1, 5, 6, 7, 8, 9, 11. Then it prompts "Enter element to search in array 8". The user enters 8, and the program outputs "element found at index 7". Finally, it prompts "Press any key to continue . . .".

```
Z:\DS\25BCA060\prog2.exe
enter array to a assending order ::
Enter Element 0= 2
Enter Element 1= 4
Enter Element 2= 3
Enter Element 3= 1
Enter Element 4= 5
Enter Element 5= 6
Enter Element 6= 7
Enter Element 7= 8
Enter Element 8= 9
Enter Element 9= 11
Enter element to search in array 8
element found at index 7
Press any key to continue . . .
```

Program-3.// Write a program in C++ print value of variable and its address. (Using Class).

```
#include<iostream>
using namespace std;
class var{
public:
    int var;
    void input() {
        cout << "Enter a value = ";
        cin >> var;
    }
    void display() {
        cout << "value of variable = " << var << endl;
        cout << "Address of variable is = " << &var << endl;
    }
};
int main() {
    var v;
    v.input();
    v.display();
    system("pause");
    return 0;
}
```

}

OUTPUT:


```

Z:\DS\258CA060\prog3.exe
Enter a value = 4
value of variable = 4
Address of variable is = 0x7afe4c
Press any key to continue . . .

```

Program-4.//Write program to implement operations Create and display using Singly link list.

```

#include<iostream>
using namespace std;
class list{
public:
    int data;
    list *link;
};
List *start2 = NULL;
list *start = NULL;
list *temp = NULL;
void creation(){
    int n,i,d;
    cout << "Enter number of nodes to be created= ";
    cin >>n;
    i=0;
    while(i!=n) {
        if(n>0) {
            if(i==0) {
                cout << "node number: " << (i+1) << endl;
                list *newnode = new list;
                cout << "Enter the data to be entered= ";
                cin >> d;
                newnode->data=d;
                newnode->link=NULL;
                temp = newnode;
                start = newnode;
            }
            else {
                cout << "node number: " << (i+1) << endl;
                list *newnode = new list;
                cout << "Enter the data to be entered= ";
                cin >> d;
                newnode->data=d;
                newnode->link=NULL;
                temp->link=newnode;
                temp=newnode;
            }
        }
        i++;
    }
    cout << "start : "<<(start)<<" :: temp "<<(temp)<< endl;
}
void display() {
    temp=start;
    int i=1;

```

```

    cout << "\noutput:: " << endl;
    while(temp!= NULL) {
        //cout << "data of node" << i << " : " << temp->data << endl;
        cout << temp->data << "-> " ;
        i++;
        temp = temp->link;
    }
    cout << "0"<< endl;
}
int main(){
    int s;
    creation();
    cout << "first list ::: " << endl;
    display();
    while(true) {
        cout << "1.insert at begin" << endl;
        cout << "2.delete at last node"<< endl;
        cout << "3.delete at begin node"<< endl;
        cout << "4.insert at Last " << endl;
        cout << "5.split the list " << endl;
        cout << "6.merge the list " << endl;
        cout << "7.add insert after " << endl;
        cout << "8.add insert before " << endl;
        cout << "9. node counting " << endl;
        cout << endl;
        cout << "select your choice no. :::";
        cin >> s;
        system("pause");
        return 0;
    }
}

```

OUTPUT:

```

Z:\DS\25BCA060\prog4.exe
Enter number of nodes to be created= 3
node number: 1
Enter the data to be entered= 4
node number: 2
Enter the data to be entered= 5
node number: 3
Enter the data to be entered= 7
start :0x1e1430 :: temp 0x1e18b0
first list :::

output::
4-> 5-> 7-> 0
1.insert at begin
2.delete at last node
3.delete at begin node
4.insert at Last
5.split the list
6.merge the list
7.add insert after
8.add insert before
9. node counting

select your choice no. :::

```

Program-5.//Write program to implement operations insert at begin using Singly link list.

```

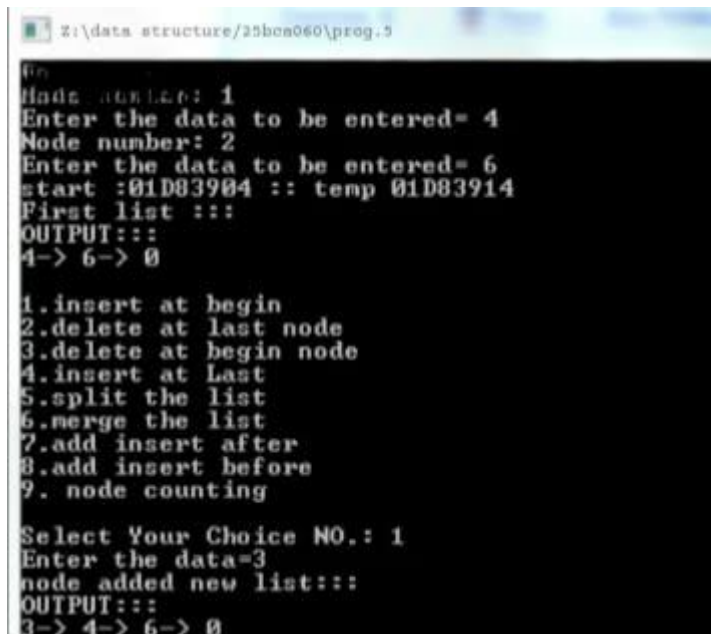
void insertAtBegin(){
    int d;

```

```

list *newnode = new list;
cout << "Enter the data=";
cin >> d;
newnode->data = d;
newnode->link=start;
start=newnode;
}
int main(){
switch (s) {
    case 1:
        insertAtBegin();
        cout << "node added new list:::" << endl;
        display();
        break;
}
    system("pause");
    return 0;
}

```

OUTPUT:


```

Z:\data structure\25ben060\prog:3
67
Node added new list: 1
Enter the data to be entered= 4
Node number: 2
Enter the data to be entered= 6
start :01D83904 :: temp 01D83914
First list :::
OUTPUT:::
4-> 6-> 0

1.insert at begin
2.delete at last node
3.delete at begin node
4.insert at Last
5.split the list
6.merge the list
7.add insert after
8.add insert before
9. node counting

Select Your Choice NO.: 1
Enter the data=3
node added new list:::
OUTPUT:::
3-> 4-> 6-> 0

```

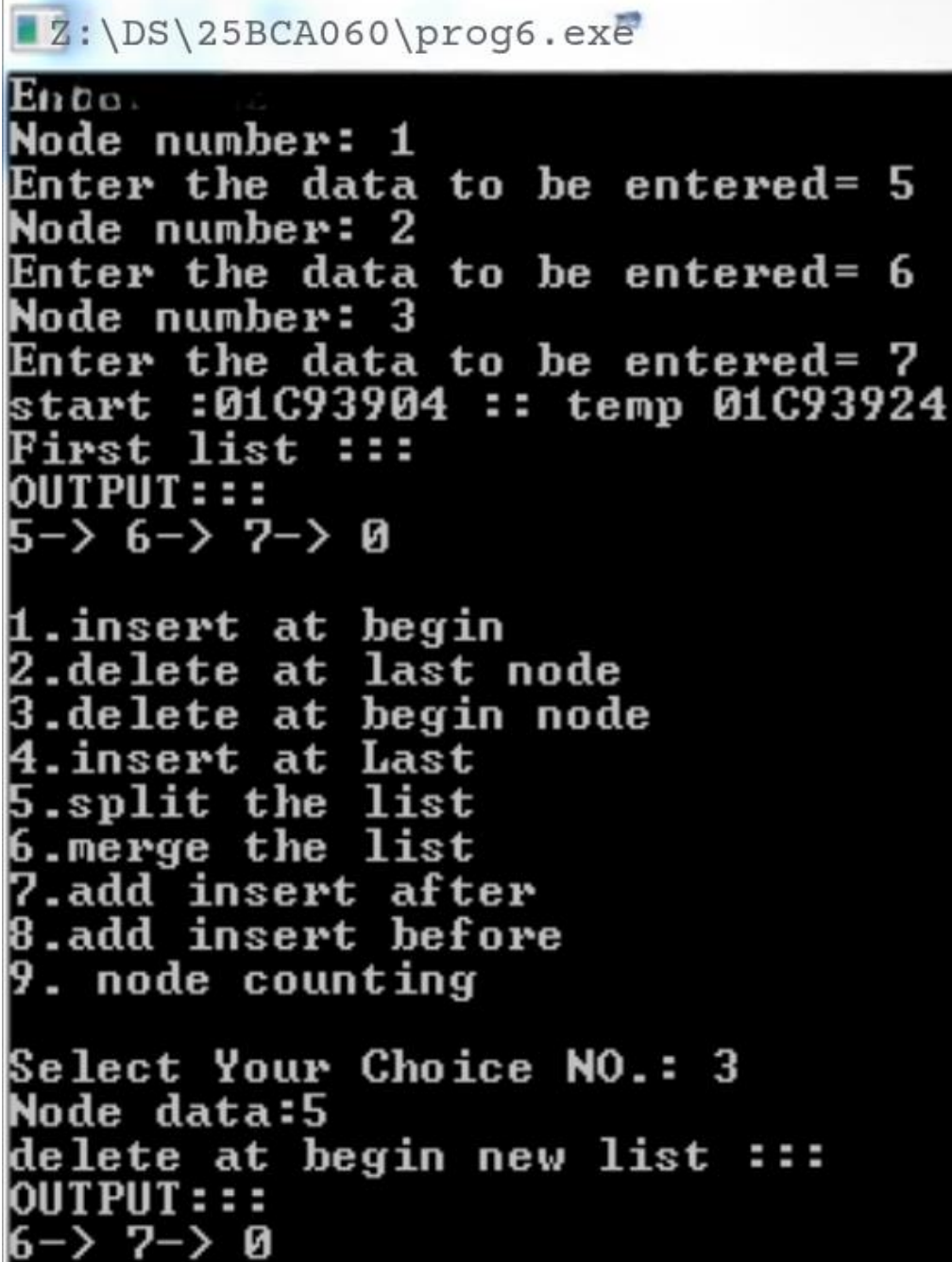
Program-6.//Write program to implement operations delete from first using Singly link list.

```

void deleteAtbegin()
{
    list *temp=start;
    start = start -> link;
    cout << "Node data:" << temp->data <<endl ;
    delete temp;
}
int main(){
case 3:
    deleteAtbegin();
    cout << "delete at begin new list :::" << endl;
    display();
    break;
}
    system("pause");
    return 0;
}

```

}

OUTPUT:


```

Z:\DS\25BCA060\prog6.exe
Enter the data to be entered= 5
Node number: 1
Enter the data to be entered= 6
Node number: 2
Enter the data to be entered= 7
Node number: 3
start :01C93904 :: temp 01C93924
First list :::
OUTPUT:::
5-> 6-> 7-> 0

1.insert at begin
2.delete at last node
3.delete at begin node
4.insert at Last
5.split the list
6.merge the list
7.add insert after
8.add insert before
9. node counting

Select Your Choice NO.: 3
Node data:5
delete at begin new list :::
OUTPUT:::
6-> 7-> 0

```

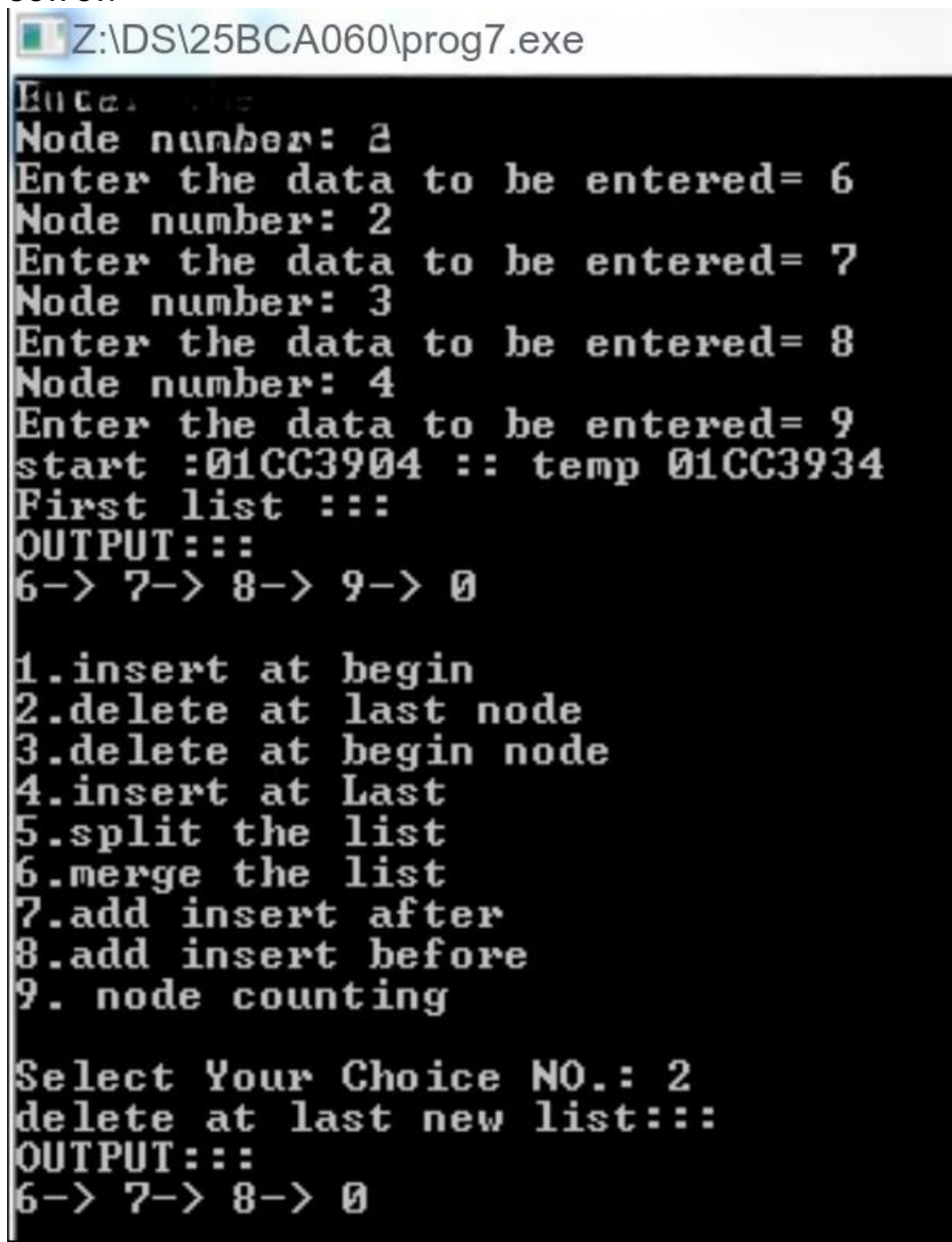
Program-7.// Write program to implement operations delete from last using Singly link list.

```

list *ptr;
void deleteAtLast() {
    if(start==NULL) {
        cout << "linked list is empty" << endl;
        return;
    }
    if(start->link==NULL) {
        delete start;
        start = NULL;
        return;
    }
    temp=start;

```

```
while(temp->link!=NULL) {  
    ptr=temp;  
    temp=temp->link;  
}  
delete temp;  
ptr->link=NULL;  
}  
int main() {  
case 2:  
    deleteAtLast();  
    cout<<"delete at last new list::" << endl;  
    display();  
    break;  
}  
system("pause");  
return 0;  
}
```

OUTPUT:

```
Z:\DS\25BCA060\prog7.exe  
Node number: 2  
Enter the data to be entered= 6  
Node number: 2  
Enter the data to be entered= 7  
Node number: 3  
Enter the data to be entered= 8  
Node number: 4  
Enter the data to be entered= 9  
start :01CC3904 :: temp 01CC3934  
First list ::  
OUTPUT::  
6-> 7-> 8-> 9-> 0  
  
1.insert at begin  
2.delete at last node  
3.delete at begin node  
4.insert at Last  
5.split the list  
6.merge the list  
7.add insert after  
8.add insert before  
9. node counting  
  
Select Your Choice NO.: 2  
delete at last new list::  
OUTPUT::  
6-> 7-> 8-> 0
```

Program-8. // Write program to implement operations insert at last, Sorting, Splitting, Merging, Counting Operations (Total no. of nodes, even and odd no. of nodes) using Singly link list.

```
void insertAtLast(){
    int d;
    list *newnode = new list;
    cout << "Enter the data=";
    cin >> d;
    newnode->data = d;
    newnode->link = NULL;
    if(start==NULL) {
        start = newnode;
        cout << "list empty; node added at fist" << endl;
        return ;
    }
    else {
        list *temp=start;
        while(temp->link!=NULL)
            temp=temp->link;
        temp->link=newnode;
    }
}

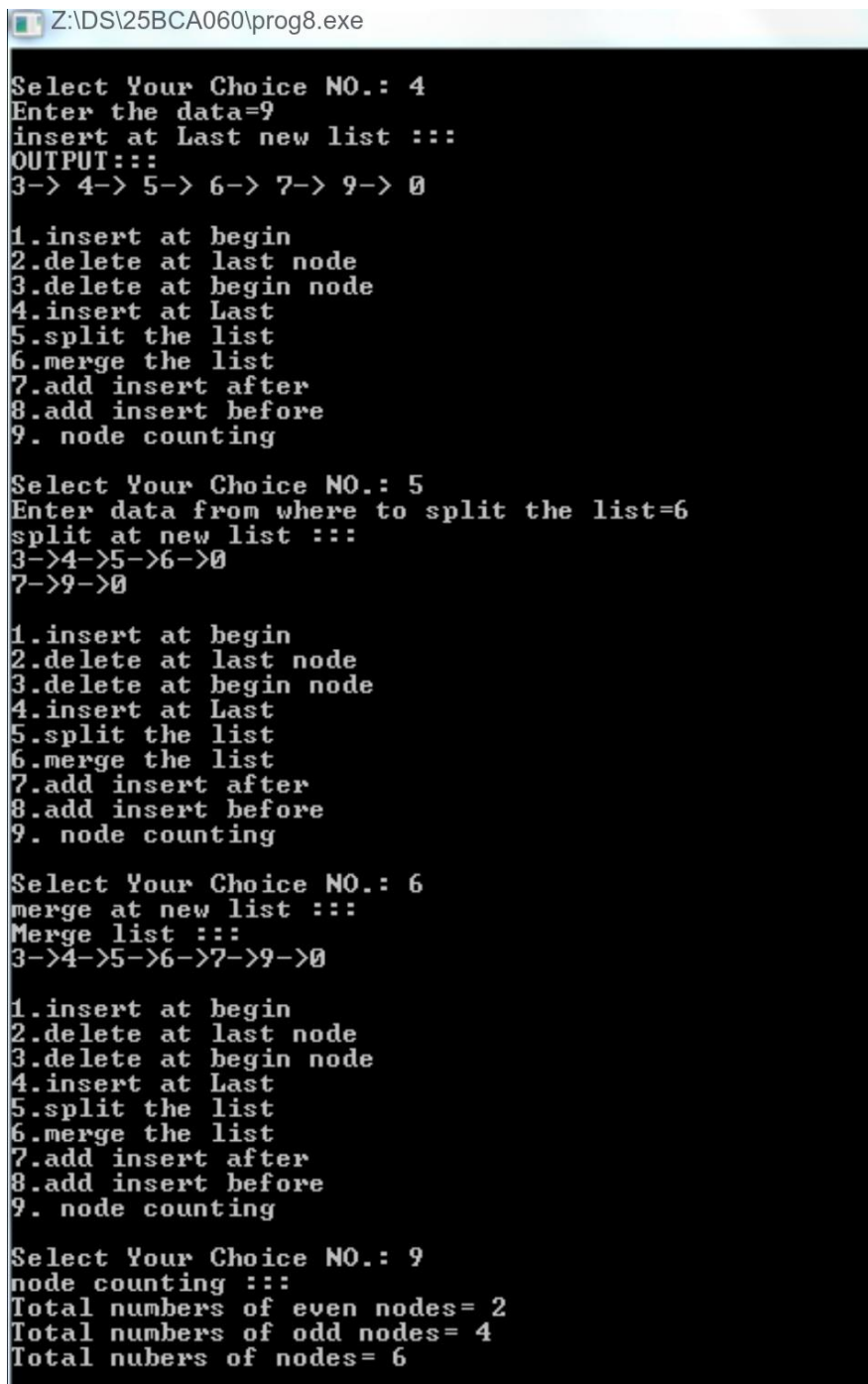
void split(){
    if(start==NULL) {
        cout << "list empty no splitting" << endl;
        return;
    }
    int target;
    cout << "Enter data from where to split the list=";
    cin >> target;
    list *cur = start;
    while(cur!=NULL && cur->data!=target) {
        cur = cur->link;
    }
    if(cur==NULL) {
        cout << "data not found at list" << endl;
        return;
    }
    if(cur->link==NULL) {
        cout << "last,node,no more element to split" << endl;
        return;
    }
    start2=cur->link;
    cur->link=NULL;
}

void splitdisplay(){
    if(start==NULL) {
        cout << "list empty" << endl;
        return;
    }
    else{
        list *p1=start;
        while(p1!=NULL){
            cout << p1->data << "->";
            p1=p1->link;
        }
    }
}
```

```
    }
    cout << "0" << endl;
}
if(start2==NULL) {
    cout << "list empty" << endl;
    return;
}
else {
    list *p2=start2;
    while(p2!=NULL) {
        cout << p2->data << "->";
        p2=p2->link;
    }
    cout << "0" << endl;
}
}
void merge(){
    if(start2==NULL) {
        cout << "list 2 empty to merge" << endl;
        return;
    }
    if(start==NULL) {
        start = start2;
        start2=NULL;
        cout << "list mergest at list 1 empty" << endl;
    }
    else {
        list *cur=start;
        while(cur->link!=NULL) {
            cur=cur->link;
        }
        cur->link=start2;
        start2=NULL;
    }
}
void mergedisplay(){
    cout << "Merge list :::" << endl;
    if(start==NULL)
        cout << "empty list merge " << endl;
    else {
        list *temp=start;
        while(temp!=NULL) {
            cout << temp->data << "->";
            temp=temp->link;
        }
        cout << "0" << endl;
    }
}
void count(){
    int even=0, odd=0;
    list *temp=start;
    while(temp!=NULL) {
        if(temp->data % 2 == 0)
            even+=1;
        else
            odd+=1;
        temp=temp->link;
    }
```

```
}
    cout << "Total numbers of even nodes= " << even << endl;
    cout << "Total numbers of odd nodes= " << odd << endl;
    cout << "Total nubers of nodes= " << (even+odd) << endl;
}
int main(){
case 4:
    insertAtLast();
    cout << "insert at Last new list :::" << endl;
    display();
    break;
case 5:
    split();
    cout << "split at new list :::" << endl;
    splitdisplay();
    break;
case 6:
    merge();
    cout << "merge at new list :::" << endl;
    mergedisplay();
    break;
case 9:
    cout << "node counting :::" << endl;
    count();
    break;
}
    system("pause");
    return 0;
}
```

OUTPUT:



```

Z:\DS\25BCA060\prog8.exe

Select Your Choice NO.: 4
Enter the data=9
insert at Last new list :::
OUTPUT:::
3-> 4-> 5-> 6-> 7-> 9-> 0

1.insert at begin
2.delete at last node
3.delete at begin node
4.insert at Last
5.split the list
6.merge the list
7.add insert after
8.add insert before
9. node counting

Select Your Choice NO.: 5
Enter data from where to split the list=6
split at new list :::
3->4->5->6->0
7->9->0

1.insert at begin
2.delete at last node
3.delete at begin node
4.insert at Last
5.split the list
6.merge the list
7.add insert after
8.add insert before
9. node counting

Select Your Choice NO.: 6
merge at new list :::
Merge list :::
3->4->5->6->7->9->0

1.insert at begin
2.delete at last node
3.delete at begin node
4.insert at Last
5.split the list
6.merge the list
7.add insert after
8.add insert before
9. node counting

Select Your Choice NO.: 9
node counting :::
Total numbers of even nodes= 2
Total numbers of odd nodes= 4
Total nubers of nodes= 6

```

Program-9.//Write program to implement operations insert at specified location (Before or After the Node) and delete any specified node using Singly link list.

```

void insertafter(){
    if(start==NULL) {
        cout << "List empty" << endl;
        return;
    }
    int target,d;
    cout << "Enter where to add after= ";
    cin >> target;
    list *ptr =start;
    while(ptr!=NULL && ptr->data!=target) {
        ptr=ptr->link;
    }
    if(ptr==NULL) {

```

```
        cout << "List empty" << endl;
        return;
    }
    else{
        list *newnode=new list;
        cout << "Enter the data= ";
        cin >> d;
        newnode->data =d;
        newnode->link=ptr->link;
        ptr->link= newnode;
    }
}

void insertbefore(){
    if(start==NULL) {
        cout << "list empty" << endl;
        return;
    }
    int target,d;
    cout << "Enter where to add before= ";
    cin >> target;
    list *temp = start;
    list *ptr =start;
    if(ptr==NULL) {
        cout << "list empty" << endl;
        return;
    }
    else {
        list *newnode = new list;
        cout << "Enter the data= ";
        cin >> d;
        newnode->data=d;
        while(ptr->data!=target) {
            temp=ptr;
            ptr=ptr->link;
        }
        newnode->link=temp->link;
        temp->link=newnode;
    }
}

int main(){
    case 7:
        insertafter();
        cout << "added insert after :::" << endl;
        display();
        break;
    case 8:
        insertbefore();
        cout << "added insert before :::" << endl;
        display();
        break;
    }
    system("pause");
    return 0;
}
```

OUTPUT:

```

Z:\DS\25BCA060\prog 9.exe
Select Your Choice NO.: 7
Enter where to add after= 6
Enter the data= 7
added insert after :::
OUTPUT:::
4-> 5-> 6-> 7-> 0

1.insert at begin
2.delete at last node
3.delete at begin node
4.insert at Last
5.split the list
6.merge the list
7.add insert after
8.add insert before
9. node counting

Select Your Choice NO.: 8
Enter where to add before= 6
Enter the data= 6
added insert before :::
OUTPUT:::
4-> 5-> 6-> 6-> 7-> 0

```

Program-10.// Write program to implement following operations using Doubly link list

//a.Insert at first

//b.Insert at Last

//c.Insert at specified location (Before or After the Node)

```

#include<iostream>
using namespace std;
class list{
public:
    int data;
    list *prev;
    list *next;
};
list *start = NULL;
list *temp = NULL;
void creation(){
    int n,i,d;
    cout << "Enter number of nodes to be created= ";
    cin >>n;
    i=0;
    while(i!=n) {
        if(n>0) {
            if(i==0) {
                cout << "Node number: " << (i+1) << endl;
                list *newnode = new list;
                cout << "Enter the data to be entered= ";
                cin >> d;
                newnode->prev=NULL;
                newnode->data=d;
                newnode->next=NULL;
                temp = newnode;
                start = newnode;
            }
            else {
                cout << "Node number: " << (i+1) << endl;
                list *newnode = new list;
                cout << "Enter the data to be entered= ";
                cin >> d;
                newnode->prev=temp->next;
                newnode->data=d;
                temp->next=newnode;
                temp=newnode;
            }
        }
        i++;
    }
}

```

```
    }  
    i++;  
}  
}  
void display(){  
    temp=start;  
    int i=1;  
    cout <<"OUTPUT::: " << endl;  
    while(temp!= NULL) {  
        cout << temp->data << "-> " ;  
        i++;  
        temp = temp->next;  
    }  
    cout << "0" << endl; }
```

//a. insert at first

```
void insertAtfirst(){  
    int d;  
    list *newnode = new list;  
    cout << "Enter the data=";  
    cin >> d;  
    if(start==NULL) {  
        newnode->prev=NULL;  
        newnode->data=d;  
        newnode->next=NULL;  
        cout << "list empty, node added at fist" << endl;  
        return ;  
    }  
    else {  
        newnode->prev=NULL;  
        newnode->data=d;  
        newnode->next=start;  
        start->prev=newnode;  
        start=newnode;  
    }  
}
```

//b. insert at last

```
void insertAtLast(){  
    int d;  
    list *newnode = new list;  
    cout << "Enter the data=";  
    cin >> d;  
    newnode->data = d;  
    newnode->next = NULL;  
    if(start==NULL) {  
        newnode->prev=NULL;  
        start = newnode;  
        cout << "list empty, node added at last" << endl;  
        return ;  
    }  
    else {  
        list *temp=start;  
        while(temp->next!=NULL)  
            temp=temp->next;  
        newnode->prev=temp;  
        temp->next=newnode;  
    }  
}
```

//c. insert at specified after and Before node

```
void insertAtbefore(){
    int target,d;
    cout << "Enter where to add before= ";
    cin >> target;
    list *temp = start;
    list *ptr =start;
    list *newnode = new list;
    if(ptr==NULL) {
        cout << "list empty" << endl;
        start=newnode;
        cout << "newnode added at first";
        return;
    }
    else {
        cout << "Enter the data= ";
        cin >> d;
        newnode->data=d;
        while(ptr->data!=target) {
            temp=ptr;
            ptr=ptr->next;
        }
        newnode->prev=temp->prev;
        newnode->next=temp->next;
        temp->next=newnode;
    }
}

void insertAtafter(){
    int target,d;
    cout << "Enter where to add after= ";
    cin >> target;
    list *newnode = new list;
    cout << "Enter the data= ";
    cin >> d;
    newnode->data =d;
    list *ptr =start;
    while(ptr!=NULL && ptr->data!=target){
        ptr=ptr->next;
    }
    if(ptr==NULL) {
        cout << "List empty" << endl;
        start=newnode;
        cout << "node added as first";
        return;
    }
    else{
        newnode->prev=ptr->prev;
        newnode->next=ptr->next;
        ptr->next= newnode;
    }
}

int main(){
    int s;
    creation();
    cout << "First created list :::" << endl;
    display();
    while(true) {
```

```
cout << "-----"<<endl;
cout << "        SELECT        "<< endl;
cout << "-----"<<endl;
cout << "1.insert at first node "<< endl;
cout << "2.insert at last node "<< endl;
cout << "3.insert before a specific node "<< endl;
cout << "4.insert after a specific node "<< endl;
cout << "5.delete at last node "<< endl;
cout << "6.delete at first node "<< endl;
cout << "7.delete specific node "<< endl;
cout << "-----"<<endl;
cout << "Select Your Choice NO.: ";
cin >>s;
cout << "-----"<<endl;
switch (s) {
    case 1:
        insertAtfirst();
        cout << "Node added new list :::" << endl;
        display();
        break;
    case 2:
        insertAtLast();
        cout << "Node added at last new list :::" << endl;
        display();
        break;
    case 3:
        insertAtbefore();
        cout << "Insert before a specific node :::" << endl;
        display();
        break;
    case 4:
        insertAtafter();
        cout << "Insert after a specific node :::" << endl;
        display();
        break;
    default:
        cout << "invalid value" << endl;
        break;
}
}
system("pause");
return 0;
}
```

OUTPUT:

```

Z:\DS\25BCA060prog10.exe
Enter number of nodes to be created= 5
Node number: 1
Enter the data to be entered= 3
Node number: 2
Enter the data to be entered= 4
Node number: 3
Enter the data to be entered= 5
Node number: 4
Enter the data to be entered= 6
Node number: 5
Enter the data to be entered= 7
First created list :::
OUTPUT:::
3-> 4-> 5-> 6-> 7-> 0

```

//insert at last and first

```

Z:\ds\25bca0060\prog10.exe
-----
                SELECT
-----
1.insert at first node
2.insert at last node
3.insert before a specific node
4.insert after a specific node
5.delete at last node
6.delete at first node
7.delete specific node
-----
Select Your Choice NO.: 1
-----
Enter the data=2
Node added new list :::
OUTPUT:::
2-> 3-> 4-> 5-> 6-> 7-> 0
-----
                SELECT
-----
1.insert at first node
2.insert at last node
3.insert before a specific node
4.insert after a specific node
5.delete at last node
6.delete at first node
7.delete specific node
-----
Select Your Choice NO.: 2
-----
Enter the data=9
Node added at last new list :::
OUTPUT:::
2-> 3-> 4-> 5-> 6-> 7-> 9-> 0

```

//insert after and before

```

Z:\ds\25bca060\progl0.exe
-----
Select Your Choice NO.: 3
-----
Enter where to add before= 5
Enter the data= 12
Insert before a specific node :::
OUTPUT:::
2-> 3-> 4-> 12-> 5-> 6-> 7-> 9-> 0
-----
                        SELECT
-----
1.insert at first node
2.insert at last node
3.insert before a specific node
4.insert after a specific node
5.delete at last node
6.delete at first node
7.delete specific node
-----
Select Your Choice NO.: 4
-----
Enter where to add after= 7
Enter the data= 16
Insert after a specific node :::
OUTPUT:::
2-> 3-> 4-> 12-> 5-> 6-> 7-> 16-> 9-> 0

```

Program-11.//Write program to implement following operations using Doubly link list

a. Delete from first , b. Delete from last , c. Delete any specified node

//b.delete at last

```

void deleteAtLast(){
    list *ptr;
    temp=start;
    if(start==NULL) {
        cout << "linked list is empty" << endl;
        return;
    }
    if(start->next==NULL) {
        delete start;
        start = NULL;
        return;
    }
    list *temp=start;
    while(temp->next!=NULL) {
        ptr=temp;
        temp=temp->next;
    }
    delete temp;
    ptr->prev=temp->next;
}

```

```
    ptr->next=NULL;
}
//a.delete at first
void deleteAtfirst(){
    list *temp=start;
    start -> prev = NULL;
    start = start -> next;
    cout << "Delete node data:" << temp->data <<endl ;
    delete (temp);
}
//c.delete at specific
void deletespecific(){
    list *ptr=start;
    int target;
    cout << "Enter where to delete node = ";
    cin >> target;
    while(ptr->data!=target) {
        temp = ptr;
        ptr=ptr->next;
    }
    temp->prev=ptr->prev;
    temp->next=ptr->next;
    delete(ptr);
}
int main(){
    case 5:
        deleteAtLast();
        cout << "Delete at last new list :::" << endl;
        display();
        break;
    case 6:
        deleteAtfirst();
        cout << "Delete at first new list :::" << endl;
        display();
        break;
    case 7:
        deletespecific();
        cout << "Delete specific node :::" << endl;
        display();
        break;
    default:
        cout << "invalid value" << endl;
        break;
}
}
system("pause");
return 0;
}
```

OUTPUT:

Z:\DS\25BCA060\progl1.exe

Select Your Choice NO.: 5

Delete at last new list :::

OUTPUT:::

2-> 3-> 4-> 5-> Ø

SELECT

1.insert at first node

2.insert at last node

3.insert before a specific node

4.insert after a specific node

5.delete at last node

6.delete at first node

7.delete specific node

Select Your Choice NO.: 6

Delete node data:2

Delete at first new list :::

OUTPUT:::

3-> 4-> 5-> Ø

SELECT

1.insert at first node

2.insert at last node

3.insert before a specific node

4.insert after a specific node

5.delete at last node

6.delete at first node

7.delete specific node

Select Your Choice NO.: 7

Enter where to delete node = 4

Delete specific node :::

OUTPUT:::

3-> 5-> Ø

Program-12.//Write a program to implement bubble sort.

```
#include<iostream>
using namespace std;
int main() {
    int arr[10],i,j,temp;
    cout << " ::: Bubble sort ::: " << endl;
    for(i=0;i<10;i++) {
        cout << "Enter element " << (i+1) << " = ";
        cin >> arr[i];
    }
    cout << endl;
    cout << "Array element ::: " << endl;
    for(i=0;i<10;i++) {
        cout << "a[" << i << "] = " << arr[i] << ", ";
    }
}
```

```

for(i=0;i<10;i++) {
    for(j=0;j<10;j++) {
        if(arr[j]>arr[j+1]) {
            temp=arr[j];
            arr[j]=arr[j+1];
            arr[j+1]=temp;
        }
    }
}
cout << endl;
cout << "Sorted array element ::: " << endl;
for(i=0;i<10;i++) {
    cout << "a["<< i << "] = " << arr[i] << ", ";
}
system("pause");
return 0;
}

```

OUTPUT:

```

Z:\DS\25BCA060\prog12.exe
::: Bubble sort :::
Enter element 1 = 5
Enter element 2 = 37
Enter element 3 = 69
Enter element 4 = 80
Enter element 5 = 90
Enter element 6 = 76
Enter element 7 = 51
Enter element 8 = 21
Enter element 9 = 17
Enter element 10 = 9

Array element :::
a[0] = 5,a[1] = 37,a[2] = 69,a[3] = 80,a[4] = 90,a[5] = 76,a[6] = 51,a[7] = 21,a[8] = 17,a[9] = 9,
Sorted array element :::
a[0] = 5,a[1] = 9,a[2] = 17,a[3] = 21,a[4] = 26,a[5] = 37,a[6] = 51,a[7] = 69,a[8] = 76,a[9] = 80,Press any key to continue

```

Program-13.//Write a program to implement insertion sort.

```

#include<iostream>
using namespace std;
int main() {
    int arr[10],i,j,k,temp;
    cout << " ::: INSERTION SORT ::: " << endl;
    cout << "Enter array element:::" << endl;
    for(i=0;i<10;i++) {
        cout << "Enter element " << (i+1) << " = ";
        cin >> arr[i];
    }
    cout << endl;
    cout << "Array element ::: " << endl;
    for(i=0;i<10;i++) {
        cout << "a["<< i << "] = " << arr[i] << ", ";
    }
    for(k=1;k<10;k++) {
        temp=arr[k];
        j=k-1;
        while(temp<=arr[j]) {
            arr[j+1]=arr[j];

```

```

    j--;
    if(j==1) {
        break;
    }
}
arr[j+1]=temp;
}
cout << endl;
cout << "Sorted array element :::" << endl;
for(i=0;i<10;i++) {
    cout << "a[" << i << "] = " << arr[i] << ", ";
}
system("pause");
return 0;
}

```

OUTPUT:

```

Z:\DS\25BCA060\prog13.exe
::: INSERTION SORT :::
Enter array element:::
Enter element 1 = 6
Enter element 2 = 54
Enter element 3 = 1
Enter element 4 = 43
Enter element 5 = 29
Enter element 6 = 80
Enter element 7 = 96
Enter element 8 = 52
Enter element 9 = 12
Enter element 10 = 9

Array element :::
a[0] = 6,a[1] = 54,a[2] = 1,a[3] = 43,a[4] = 29,a[5] = 80,a[6] = 96,a[7] = 52,a[8] = 12,a[9] = 9,
Sorted array element :::
a[0] = 1,a[1] = 6,a[2] = 9,a[3] = 12,a[4] = 29,a[5] = 43,a[6] = 52,a[7] = 54,a[8] = 80,a[9] = 96,Press any key to continue . . .

```

Program-14.//Write a program to implement selection sort.

```

#include<iostream>
using namespace std;
int i,j,arr[10],index,temp;
void selectionsort() {
    for(i=0;i<10;i++) {
        index=i;
        for(j=i+1;j<10;j++) {
            if(arr[j]<arr[index])
                index=j;
        }
        temp=arr[i];
        arr[i]=arr[index];
        arr[index]=temp;
    }
}
int main(){
    cout << "SELECTION SORT:::" << endl;
    for(i=0;i<10;i++) {
        cout << "Enter element " <<(i+1) << " = ";
        cin >> arr[i];
    }
    cout << "array elemet::: " << endl;
    for(i=0;i<10;i++) {
        cout << "a[" << i << "] = " << arr[i] << ", ";
    }
}

```

```

    }
    selectionsort();
    cout << endl;
    cout << "sorted element " << endl;
    for(i=0;i<10;i++) {
        cout << "a[" << i << "] = " << arr[i] << ", ";
    }
    system("pause");
    return 0;
}

```

OUTPUT:

```

Z:\DS\25BCA060\prog14.exe
SELECTION SORT:::
enter element 1= 62
enter element 2= 12
enter element 3= 19
enter element 4= 83
enter element 5= 11
enter element 6= 3
enter element 7= 9
enter element 8= 21
enter element 9= 31
enter element 10= 15
array element:::
a[0] = 62,a[1] = 12,a[2] = 19,a[3] = 83,a[4] = 11,a[5] = 3,a[6] = 9,a[7] = 21,a[8] = 31,a[9] = 15,
sorted element
a[0] = 3,a[1] = 9,a[2] = 11,a[3] = 12,a[4] = 15,a[5] = 19,a[6] = 21,a[7] = 31,a[8] = 62,a[9] = 83,Press any key to continue . . .

```

Program-15.//Write a program to implement quick sort.

```

#include<iostream>
using namespace std;
int beg=0,end=0,temp,loc;
int i,j,arr[10];
void partition(int b, int e) {
    int left=b,right=e,f=0;
    loc=b;
    while(f==0) {
        while((arr[loc]<=arr[right])&&loc!=right) {
            right--;
        }
        if(loc==right)
            f=1;
        else if(arr[loc] > arr[right])
        {
            temp=arr[loc];
            arr[loc]=arr[right];
            arr[right]=temp;
            loc=right;
        }
        if(f==0) {
            while((arr[loc] >= arr[left]) && loc!=left) {
                left++;
            }
            if(loc==left)
                f=1;
            else if(arr[loc] < arr[left]) {
                temp=arr[loc];
                arr[loc]=arr[left];
                arr[left]=temp;
            }
        }
    }
}

```

```

        loc=left;
    }
}
}
}
void quicksort(int beg,int end) {
    if(beg<end) {
        partition(beg,end);
        quicksort(beg,loc-1);
        quicksort(loc+1,end);
    }
}
int main() {
    cout << " :::QUICK SORT::: " << endl;
    cout << "Enter array element::: " << endl;
    for(i=0;i<10;i++) {
        cout << "Enter element " <<(i+1) << " = ";
        cin >> arr[i];
    }
    cout << "array elemet::" << endl;
    for(i=0;i<10;i++) {
        cout << "a[" << i << "] = " << arr[i] << ", ";
    }
    quicksort(0,9);
    cout << endl;
    cout << "sorted element " << endl;
    for(i=0;i<10;i++) {
        cout << "a[" << i << "] = " << arr[i] << ", ";
    }
    system("pause");
    return 0;
}

```

OUTPUT:

```

Z:\DS\25BCA060\prog15.exe
:::QUICK SORT:::
Enter array element:::
Enter element 1= 5
Enter element 2= 12
Enter element 3= 10
Enter element 4= 29
Enter element 5= 64
Enter element 6= 32
Enter element 7= 45
Enter element 8= 61
Enter element 9= 52
Enter element 10= 22
array elemet::
a[0] = 5,a[1] = 12,a[2] = 10,a[3] = 29,a[4] = 64,a[5] = 32,a[6] = 45,a[7] = 61,a[8] = 52,a[9] = 22,
sorted element
a[0] = 5,a[1] = 10,a[2] = 12,a[3] = 22,a[4] = 29,a[5] = 32,a[6] = 45,a[7] = 52,a[8] = 61,a[9] = 64,Press any key to continue . . .

```

Program-16.//Write a program to implement merge sort.

```

#include<iostream>
using namespace std;
int beg=0,end=0,mid=0,temp[10];
int i,j,arr[10];
void mergesort(int a[],int b,int m, int e) {
    beg=b,end=e;
    int i=b, j=m+1,index=0;
    while(i<=m && j<=e) {

```

```
        if(a[i]<a[j]) {
            temp[index]=a[i];
            i++;
        }
        else {
            temp[index]=a[j];
            j++;
        }
        index++;
    }
    if(i>m) {
        while(j<=e) {
            temp[index]=a[j];
            index++;
            j++;
        }
    }
    else {
        while(i<=m) {
            temp[index]=a[i];
            index++;
            i++;
        }
    }
    int k=0;
    while(k<index) {
        a[b+k]=temp[k];
        k++;
    }
}

void merge(int a[],int b,int e) {
    if(b<e) {
        int m=(b+e)/2;
        merge(a,b,m);
        merge(a,m+1,e);
        mergesort(a,b,m,e);
    }
}

int main() {
    cout << " :::MERGE SORT::: " << endl;
    cout << "Enter array element::: " << endl;
    for(i=0;i<10;i++) {
        cout << "Enter element " <<(i+1) << " = ";
        cin >> arr[i];
    }
    cout << "array elemet:::" << endl;
    for(i=0;i<10;i++) {
        cout << "a[" << i << "] = " << arr[i] << ", ";
    }
    merge(arr,0,9);
    cout << endl;
    cout << "sorted element:::" << endl;
    for(i=0;i<10;i++) {
        cout << "a[" << i << "] = " << arr[i] << ", ";
    }
    system("pause");
    return 0;
}
```

}

OUTPUT:

```

Z:\DS\25BCA060\prog16.exe
:::MERGE SORT:::
Enter array element:::
Enter element 1= 43
Enter element 2= 21
Enter element 3= 4
Enter element 4= 32
Enter element 5= 18
Enter element 6= 79
Enter element 7= 80
Enter element 8= 54
Enter element 9= 3
Enter element 10= 9
array element:::
a[0] = 43,a[1] = 21,a[2] = 4,a[3] = 32,a[4] = 18,a[5] = 79,a[6] = 80,a[7] = 54,a[8] = 3,a[9] = 9,
sorted element:::
a[0] = 3,a[1] = 4,a[2] = 9,a[3] = 18,a[4] = 21,a[5] = 32,a[6] = 43,a[7] = 54,a[8] = 79,a[9] = 80,Press any key to continue . . .

```

Program-17. //Write a program to implement following operations(PUSH,POP,PEEP)in stack Using Array.

```

#include<iostream>
using namespace std;
int stack[100];
int top = -1,MAX;
void display(int stack[]){
    if(top == -1)
        cout << "STACK IS EMPTY" << endl;
    else {
        for(int i=0;i<=top;i++) {
            cout << stack[i] << " ";
        }
    }
    cout << endl;
}
void push(int stack[],int val) {
    if(top==MAX-1)
        cout << "STACK IS FULL" << endl;
    else {
        top++;
        stack[top]=val;
    }
}
void pop(int stack[]) {
    if(top==--1)
        cout << "STACK IS EMPTY" << endl;
    else {
        int val;
        val=stack[top];
        top--;
        cout << "Pop value is = " << val << endl;
    }
}
void peep(int stack[]) {
    if(top == NULL)
        cout << "STACK IS EMPTY" << endl;
    else {

```

```

    cout << "peep element= " << stack[top] << endl;
}
}
int main() {
    int val,s;
    cout << "Enter min stack element(1-100) = ";
    cin >> MAX;
    cout << endl;
    while(true) {
        cout << "-----" << endl;
        cout << "    SELECT    " << endl;
        cout << "-----" << endl;
        cout << "1.Push operation " << endl;
        cout << "2.Pop operation " << endl;
        cout << "3.Peep operaton " << endl;
        cout << "4.DISPLAY " << endl;
        cout << "Enter 0 to Exit " << endl;
        cout << "-----" << endl;
        cout << "Select Your Choice NO.: ";
        cin >> s;
        cout << "-----" << endl;
        cout << endl;
        switch (s) {
            case 1:
                cout << "Push operation::: " << endl;
                cout << "Enter a value to push array= ";
                cin >> val;
                push(stack,val);
                break;
            case 2:
                cout << "Pop operation::: " << endl;
                pop(stack);
                break;
            case 3:
                cout << "Peep operation::: " << endl;
                peep(stack);
                break;
            case 4:
                cout << "STACK " << endl;
                display(stack);
                break;
            default:
                cout << "invalid value" << endl;
                break;
        }
    }
    system("pause");
    return 0;
}

```

OUTPUT:**//PUSH OPERATION:****//POP OPERATION:**

```

Z:\DS\25BCA060\prog17.exe
Enter min stack element(1-100) = 5

-----
          SELECT
-----
1.Push operation
2.Pop operation
3.Peep operaton
4.DISPLAY
Enter 0 to Exit

-----
Select Your Choice NO.: 1
-----

Push operation::
Enter a value to push array= 21

```

```

Select Z:\DS\25BCA060\prog17.exe
-----
Select Your Choice NO.: 2
-----

Pop operation::
Pop value is = 21
-----
          SELECT
-----
1.Push operation
2.Pop operation
3.Peep operaton
4.DISPLAY
Enter 0 to Exit

-----
Select Your Choice NO.: 4
-----

STACK
STACK IS EMPTY

```

//PEEP OPERATION:

```

Select Z:\DS\25BCA060\prog17.exe
-----
Select Your Choice NO.: 4
-----

STACK
STACK IS EMPTY

-----
          SELECT
-----
1.Push operation
2.Pop operation
3.Peep operaton
4.DISPLAY
Enter 0 to Exit

-----
Select Your Choice NO.: 3
-----

Peep operation::
peep element= 0

```

Program-18.//Write a program to implement following operations (PUSH,POP,PEEP ,CHANGE)in stack Using Linked List.

```

#include<iostream>
using namespace std;
class node {
public:
    int data;
    node *next;
};
node *top = NULL;
void push(int value) {
    node *newnode = new node;
    newnode->data=value;
    newnode->next=top;
    top=newnode;
}
void display() {

```

```
    if(top==NULL)
        cout << "STACK IS EMPTY" << endl;
    node *temp=top;
    while(temp!=NULL) {
        cout << temp->data << "->";
        temp=temp->next;
    }
    cout << "0" << endl;
}

void pop() {
    if(top==NULL)
        cout << "Stack is empty" << endl;
    else {
        node *temp=top;
        cout << "Pop value is = " << temp->data << endl;
        top=top->next;
        delete temp;
    }
}

void peep() {
    if(top == NULL)
        cout << "STACK IS EMPTY" << endl;
    else {
        cout << "Top element= " << top->data << endl;
    }
}

void change() {
    int target,d;
    cout << "Enter data which to change= ";
    cin >> target;
    node *temp=top;
    while(temp!=NULL && temp->data!=target) {
        temp=temp->next;
    }
    if(temp==NULL) {
        cout << "List empty" << endl;
        return;
    }
    else {
        cout << "Enter the data to change= ";
        cin >> d;
        temp->data=d;
    }
}

int main() {
    int value,s;
    cout << endl;
    while(true) {
        cout << "-----" << endl;
        cout << "    SELECT    " << endl;
        cout << "-----" << endl;
        cout << "1.Push operation " << endl;
        cout << "2.Pop operation " << endl;
        cout << "3.Peep operaton " << endl;
        cout << "4.DISPLAY " << endl;
        cout << "5.Change " << endl;
        cout << "Enter 0 to Exit " << endl;
```

```
cout << "-----"<<endl;
cout << "Select Your Choice NO.: ";
cin >>s;
cout << "-----"<<endl;
cout << endl;
switch (s) {
    case 1:
        cout << "Push operation::: " << endl;
        cout << "Enter a value to push= " ;
        cin >> value;
        push(value);
        break;
    case 2:
        cout << "Pop operation::: " << endl;
        pop();
        break;
    case 3:
        cout << "Peep operation::: " << endl;
        peep();
        break;
    case 4:
        cout << "STACK "<< endl;
        display();
        break;
    case 5:
        cout << "change operation::: " << endl;
        change();
        break;
    default:
        cout << "invalid value" << endl;
        break;
}
}
system("pause");
return 0;
}
```

OUTPUT:**//PUSH OPERATION:****//POP and PEEP OPERATIONS:**

```

Z:\DS\25BCA060\prog18.exe
-----
          SELECT
-----
1.Push operation
2.Pop operation
3.Peep operation
4.DISPLAY
5.Change
Enter 0 to Exit
-----
Select Your Choice NO.: 1
-----
Push operation:::
Enter a value to push= 14

```

```

Z:\DS\25BCA060\prog18.exe
select Your Choice NO.: 2
-----
          SELECT
-----
1.Push operation
2.Pop operation
3.Peep operation
4.DISPLAY
5.Change
Enter 0 to Exit
-----
select Your Choice NO.: 3
-----
Peep operation:::
STACK IS EMPTY

```

//DISPLAY AND CHANGE:

```

Z:\DS\25BCA060\prog18.exe
Select Your Choice NO.: 5
-----
change operation:::
Enter data which to change= 13
List empty
-----
          SELECT
-----
1.Push operation
2.Pop operation
3.Peep operation
4.DISPLAY
5.Change
Enter 0 to Exit
-----
Select Your Choice NO.: 4
-----
STACK
STACK IS EMPTY
0

```

Program-19.//Write a program to implement Evaluation of given postfix expression.

```

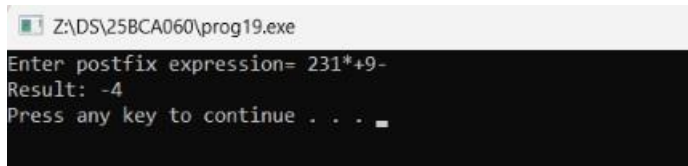
#include<iostream>
using namespace std;
int main() {
    char exp[30];
    int stack[30],top=-1,i;
    cout << "Enter postfix expression= ";
    cin >> exp;
    for(i=0;exp[i]!='\0';i++) {
        char ch=exp[i];
        if(ch>='0' && ch<='9') {

```

```

    stack[++top] = ch-'0';
}
else {
    int b=stack[top--];
    int a=stack[top--];
    switch(ch) {
        case '+':
            stack[++top]=a+b;
            break;
        case '-':
            stack[++top]=a-b;
            break;
        case '*':
            stack[++top]=a*b;
            break;
        case '/':
            stack[++top]=a/b;
            break;
    }
}
}
cout << "Result: " << stack[top] << endl;
system("pause");
return 0;
}

```

OUTPUT:

Program-20.//Write a program to implement conversion of infix expression into postfix expression (parentheses and non parentheses).

```

#include<iostream>
using namespace std;
char stack[100];
int top=-1,MAX;
bool isAlphanumeric(char ch) {
    return(ch>='0' && ch<='9' || ch>='A' && ch<='Z' || ch>='a' && ch<='z');
}
int priority(char v) {
    if(v=='+' || v=='-')
        return 0;
    if(v=='*' || v=='/')
        return 1;
    return -1;
}
void push(char stack[],char v) {
    stack[++top]=v;
}
char pop(char stack[]) {
    char v=' ';
    if(top== -1)
        cout << "stack empty" << endl;
}

```

```

else
    v= stack[top--];
return v;
}
void change(char source[],char target[]) {
    int i=0,j=0;
    strcpy(target,"");
    while(source[i] !='\0') {
        if(source[i] == '(') {
            push(stack,source[i]);
            i++;
        }
        else if(source[i] == ')') {
            while((top!=-1) && (stack[top]!='(')) {
                target[j]=pop(stack);
                j++;
            }
            if(top==--1)
                cout << "Incorrect Expression" << endl;
            pop(stack);
            i++;
        }
        else if(isAlphanumeric(source[i])) {
            target[j]=source[i];
            j++;
            i++;
        }
        else if(source[i]=='+' || source[i]=='-' || source[i]=='*' || source[i]=='/' || source[i]=='%') {
            while((top!=-1) && (stack[top]!='(') && (priority(stack[top]) > priority(source[i]))) {
                target[j]=pop(stack);
                j++;
            }
            push(stack,source[i]);
            i++;
        }
        else {
            cout << "Element incorrect in expression " << endl;
            return;
        }
    }
    while((top!=-1) && (stack[top]!='(')) {
        target[j]=pop(stack);
        j++;
    }
    target[j]='\0';
}
int main() {
    char i[100],p[100];
    cout << "Enter min stack elements = ";
    cin >> MAX;
    cout << "Enter expression= ";
    cin >> i;
    change(i,p);
    cout << "Answer = " << p << endl;
    system("pause");
    return 0;
}

```

OUTPUT:

```

Z:\DS\25BCA060\prog20.exe
Enter min stack elements = 100
Enter expression= (a+b)-(c*d)
Answer = ab+cd*-
Press any key to continue . . .

```

```

Z:\DS\25BCA060\prog20.exe
Enter min stack elements = 100
Enter expression= a-b+c*d
Answer = abcd*+-
Press any key to continue . . .

```

Program-21.//Write a program to implement recursion.

```

#include<iostream>
using namespace std;
int Fibonacci(int n) {
    if (n==0)
        return 0;
    else if (n==1)
        return 1;
    else
        return (Fibonacci(n-1) + Fibonacci(n-2));
}
int main() {
    int n,i=0,res;
    cout << "Enter the number of terms=" << endl;
    cin >> n;
    cout << "Fibonacci series:" << endl;
    for(i=0; i<n; i++) {
        res = Fibonacci(i);
        cout << res << " ";
    }
    cout << endl;
    system("pause");
    return 0;
}

```

OUTPUT:

```

Z:\DS\25BCA060\prog21.exe
Enter the number of terms=
10
Fibonacci series:
0 1 1 2 3 5 8 13 21 34
Press any key to continue . . .

```

Program-22.//Write a program to reverse the string using the stack.

```

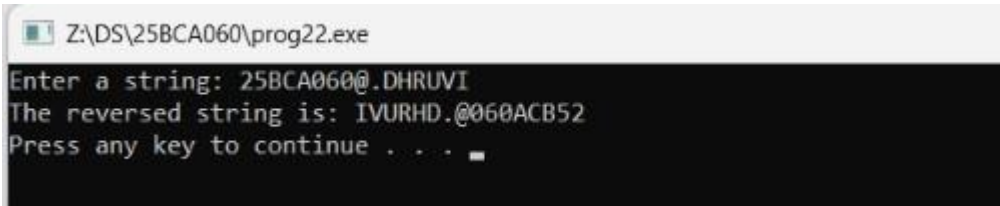
#include<iostream>
using namespace std;
int main() {
    char str[100],stack[100];

```

```

int top=-1;
cout << "Enter a string: ";
cin >> str;
for(int i=0;str[i]!='\0';i++) {
    stack[++top]=str[i];
}
cout << "The reversed string is: ";
while(top>=0) {
    cout << stack[top];
    top--;
}
cout << endl;
system("pause");
return 0;
}

```

OUTPUT:


```

Z:\DS\25BCA060\prog22.exe
Enter a string: 25BCA060@.DHRUVI
The reversed string is: IVURHD.@060ACB52
Press any key to continue . . .

```

Program-23.//Write a program to implement Simple Queue operations using Linked List.

//• ENQUEUE

//• DEQUEUE

//• Traversal (display)

```

#include<iostream>
using namespace std;
class node{
public:
    int data;
    node *next;
};
node *front=NULL;
node *rear=NULL;
void Enqueue(){
    int val;
    node *newnode = new node;
    cout << "Enter data= ";
    cin >> val;
    newnode->data = val;
    if(front == NULL) {
        front = rear = newnode;
        front->next=rear->next=NULL;
    }
    else{
        rear->next=newnode;
        rear=newnode;
        rear->next =NULL;
    }
}
void Dequeue(){
    if(front == NULL) {
        cout << "queue is empty" << endl;

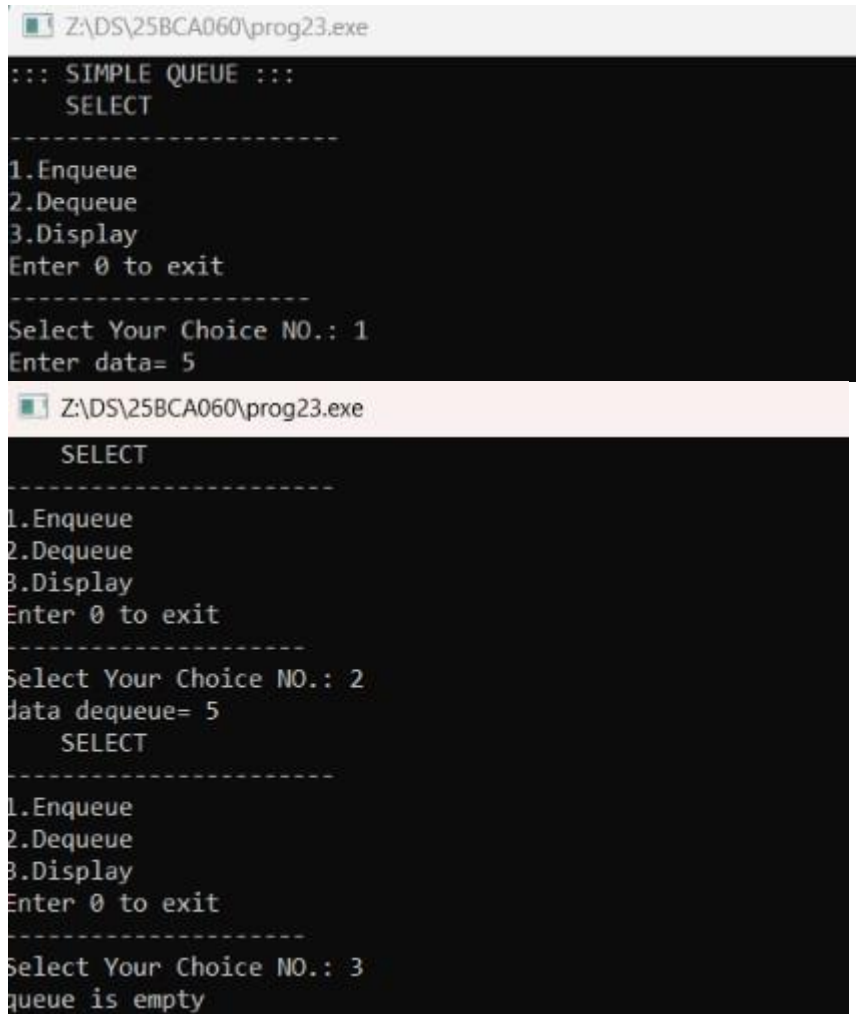
```

```

    return;
}
node *temp=front;
cout << "data dequeue= " << temp->data << endl;
front=front->next;
delete temp;
}
void display(){
    if(front == NULL) {
        cout << "queue is empty" << endl;
        return;
    }
    node *temp=front;
    cout << "queue:::" << endl;
    while(temp!=NULL) {
        cout << temp->data << "->";
        temp=temp->next;
    }
    cout << " 0 ";
}
int main(){
    int s;
    cout << " ::: SIMPLE QUEUE ::: " << endl;
    while(true) {
        cout << "  SELECT  " << endl;
        cout << "-----" << endl;
        cout << "1.Enqueue " << endl;
        cout << "2.Dequeue " << endl;
        cout << "3.Display " << endl;
        cout << "Enter 0 to exit" << endl;
        cout << "-----" << endl;
        cout << "Select Your Choice NO.: ";
        cin >> s;
        switch (s) {
            case 1:
                Enqueue();
                break;
            case 2:
                Dequeue();
                break;
            case 3:
                display();
                break;
            case 4:
                display();
                break;
            default:
                cout << "invalid value" << endl;
                break;
        }
    }
    system("pause");
    return 0;
}

```

OUTPUT:



```

Z:\DS\25BCA060\prog23.exe
::: SIMPLE QUEUE :::
  SELECT
  -----
1.Enqueue
2.Dequeue
3.Display
Enter 0 to exit
  -----
Select Your Choice NO.: 1
Enter data= 5

Z:\DS\25BCA060\prog23.exe
  SELECT
  -----
1.Enqueue
2.Dequeue
3.Display
Enter 0 to exit
  -----
Select Your Choice NO.: 2
data dequeue= 5
  SELECT
  -----
1.Enqueue
2.Dequeue
3.Display
Enter 0 to exit
  -----
Select Your Choice NO.: 3
queue is empty

```

Program-24.//Write a program to implement Circular Queue operations Using Linked List.

//• ENQUEUE

//• DQUEUE

//• Traversal (display)

```

#include<iostream>
using namespace std;
class node{
public:
    int data;
    node *next;
};
node *front = NULL;
node *rear = NULL;
void Enqueue(){
    int val;
    node *newnode = new node;
    cout << "Enter data= ";
    cin >> val;
    newnode->data = val;
    if(front == NULL) {
        front = rear = newnode;
        rear->next=front;
    }
    else{
        rear->next=newnode;
        rear=newnode;
    }
}

```

```
        rear->next =front;
    }
}

void Dequeue(){
    if(front == NULL) {
        cout << "queue is empty" << endl;
    }
    node *temp=front;
    if(front==rear) {
        cout << front->data <<"dequeue from queue" << endl;
        delete front;
        front =rear=NULL;
    }
    else{
        cout << temp->data << " dequeue from queue " << endl;
        front=front->next;
        rear->next=front;
        delete temp;
    }
}

void display(){
    if(front == NULL) {
        cout << "queue is empty" << endl;
        return;
    }
    node *temp=front;
    cout << "queue:::" << endl;
    do {
        temp=temp->next;
        cout << temp->data << "->";
    }
    while(temp!=front);
    cout << "(back to " << front->data << ")" << endl;
}

int main() {
    int s;
    cout << "::: CIRCULAR QUEUE :::" << endl;
    while(true) {
        cout << "  SELECT  " << endl;
        cout <<"-----"<<endl;
        cout << "1.Enqueue " << endl;
        cout <<"2.Dequeue "<< endl;
        cout <<"3.Display "<< endl;
        cout <<"Enter 0 to exit" << endl;
        cout <<"-----"<< endl;
        cout << "Select Your Choice NO.: ";
        cin >>s;
        switch (s) {
            case 1:
                Enqueue();
                break;
            case 2:
                Dequeue();
                break;
            case 3:
                display();
                break;
```

```

    case 4:
        display();
        break;
    default:
        cout << "invalid value" << endl;
        break;
    }
}
system("pause");
return 0;
}

```

OUTPUT:

```

Z:\DS\25BCA060\prog24.exe
::: CIRCULAR QUEUE :::
SELECT
-----
1.Enqueue
2.Dequeue
3.Display
Enter 0 to exit
-----
Select Your Choice NO.: 1
Enter data= 6

```

```

Z:\DS\25BCA060\prog24.exe
-----
Select Your Choice NO.: 2
6dequeue from queue
SELECT
-----
1.Enqueue
2.Dequeue
3.Display
Enter 0 to exit
-----
Select Your Choice NO.: 3
queue is empty

```

//25. Write a program to implement following operations on Binary Search Tree using Linked List.

// • Creation , • Insertion , • Traversal(In-order, Pre-order, Post-order)

```

#include<iostream>
using namespace std;
class node{
public:
    node *left;
    int data;
    node *right;
};
node *create(int val){
    node *newnode = new node;
    newnode->data=val;
    newnode->left=NULL;
    newnode->right=NULL;
    return newnode;
}
node *insert(node *ptr,int val){
    if(ptr == NULL){
        ptr=create(val);
    }
    else if(val < ptr->data)
        ptr->left=insert(ptr->left,val);
    else
        ptr->right=insert(ptr->right,val);
    return ptr;
}
void preorder(node *ptr){
    if(ptr == NULL)return;

```

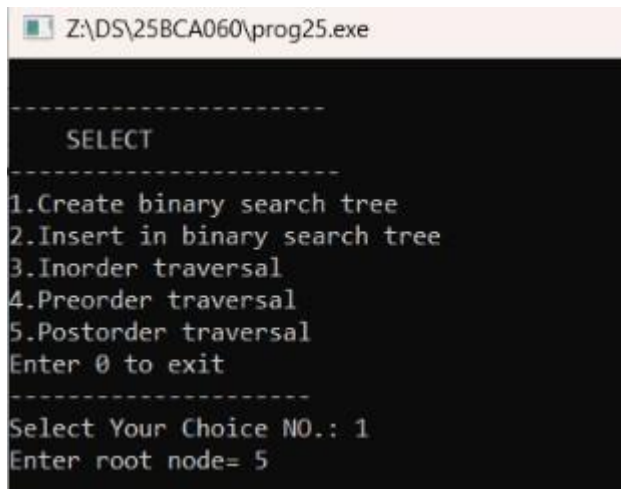
```
else{
    cout << ptr->data << " ";
    preorder(ptr->left);
    preorder(ptr->right);
}
}

void postorder(node *ptr){
    if(ptr==NULL)return;
    else{
        cout<<ptr->data <<" ";
        postorder(ptr->right);
        postorder(ptr->left);
    }
}

void inorder(node *ptr){
    if(ptr==NULL)return;
    else{
        inorder(ptr->left);
        cout << ptr->data << " ";
        inorder(ptr->right);
    }
}

int main(){
    int val,s;
    node *root=NULL;
    while(true) {
        cout <<"\n-----" << endl;
        cout << "  SELECT  " << endl;
        cout <<"-----" << endl;
        cout << "1.Create binary search tree " << endl;
        cout <<"2.Insert in binary search tree " << endl;
        cout <<"3.Inorder traversal " << endl;
        cout <<"4.Preorder traversal " << endl;
        cout <<"5.Postorder traversal " << endl;
        cout <<"Enter 0 to exit" << endl;
        cout <<"-----" << endl;
        cout << "Select Your Choice NO.: ";
        cin >>s;
        switch (s) {
            case 1:
                cout << "Enter root node= ";
                cin >>val;
                root=create(val);
                break;
            case 2:
                cout << "Enter data= ";
                cin >>val;
                insert(root,val);
                break;
            case 3:
                cout << "Inorder traversal ";
                inorder(root);
                break;
            case 4:
                cout << "Preorder traversal ";
                preorder(root);
                break;
```

```
        case 5:
            cout << "Postorder traversal ";
            postorder(root);
            break;
        default:
            cout << "INVALID INPUT" << endl;
            break;
    }
}
system("pause");
return 0;
}
```

OUTPUT:**//create**

```
Z:\DS\25BCA060\prog25.exe

-----
SELECT
-----
1.Create binary search tree
2.Insert in binary search tree
3.Inorder traversal
4.Preorder traversal
5.Postorder traversal
Enter 0 to exit
-----
Select Your Choice NO.: 1
Enter root node= 5
```

//insert

 Z:\DS\25BCA060\prog25.exe

Select Your Choice NO.: 2

Enter data= 4

SELECT-----
1.Create binary search tree

2.Insert in binary search tree

3.Inorder traversal

4.Preorder traversal

5.Postorder traversal

Enter 0 to exit

Select Your Choice NO.: 2

Enter data= 8

SELECT-----
1.Create binary search tree

2.Insert in binary search tree

3.Inorder traversal

4.Preorder traversal

5.Postorder traversal

Enter 0 to exit

Select Your Choice NO.: 2

Enter data= 67

SELECT-----
1.Create binary search tree

2.Insert in binary search tree

3.Inorder traversal

4.Preorder traversal

5.Postorder traversal

Enter 0 to exit

Select Your Choice NO.: 2

Enter data= 34

//traversal(inorder,postorder,preorder Z:\DS\25BCA060\prog25.exe

```
-----
Select Your Choice NO.: 3
Inorder traversal 4 5 8 34 67
-----
      SELECT
-----
1.Create binary search tree
2.Insert in binary search tree
3.Inorder traversal
4.Preorder traversal
5.Postorder traversal
Enter 0 to exit
-----
Select Your Choice NO.: 4
Preorder traversal 5 4 8 67 34
-----
      SELECT
-----
1.Create binary search tree
2.Insert in binary search tree
3.Inorder traversal
4.Preorder traversal
5.Postorder traversal
Enter 0 to exit
-----
Select Your Choice NO.: 5
Postorder traversal 5 8 67 34 4
-----
```